Variables, Data Structures, File I/O

Node REPL

- The Node REPL stands for:
  - Read
  - Evaluate
  - Print
  - Loop
- This is an interactive way to test JavaScript code rapidly without having to type into a file and running repeatedly.
- Can invoke on the command line by simply typing ‘node’ and the REPL will appear.
  - On Windows, it may be a separate Node.js application

Variables

JavaScript Variables

- Simply put, variables are a way to store ‘data’
- This data can be a number, string, boolean, or null
- Most programming languages are ‘strictly-typed’ languages, meaning that you must declare the type at run-time.
  - Java: String x = “Hello”;
  - C: char* x = “Hello”;
  - etc.

JavaScript Variables (cont.)

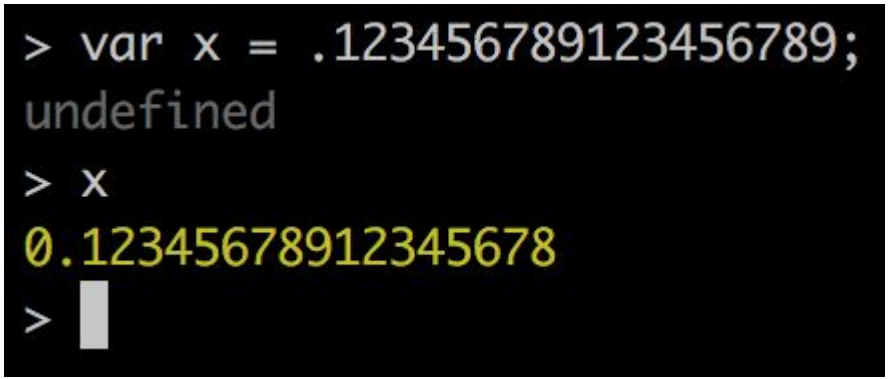
- JavaScript variables are “loosely-typed” or “dynamically-typed”
- This means you never have to explicitly state the type of the variable in the code.
  - var x = “Hello”;
  - var x = 4;
- Both examples above created two variables of different types.
- Notice the left-hand side of the assignment is the exact same for both examples.

Data Types: Number

- In most programming languages, the Number data-type is broken down into several different types.
  - Integers: Whole numbers
  - Short: Small numbers
  - Floats: Decimal numbers with low precision
  - Doubles: Decimal numbers with high-precision
  - Longs: Very large numbers
  - Etc.
- In JavaScript, all of these are simply “Numbers”

Data Types: Number

- All Numbers in JavaScript are 8-bytes, irrespective of how big, small or precise the number is (decimal values)
- 8 bits in a byte means the biggest number is (2^63) -1



Data Types: Boolean

- Is the most simple Data Type because it will always evaluate to be either:
  - true
  - false
- Booleans are 4-bytes
  - In some languages, Booleans are only 1-bit because it evaluates to either 0 or 1.

Data Types: Null

- Very similar to any other programming language. It is the data type that does not have any data.
  - Very useful when using an external API because if a specific value does not exist, then it will return null
  - Example: Location for a Twitter user - https://dev.twitter.com/overview/api/users

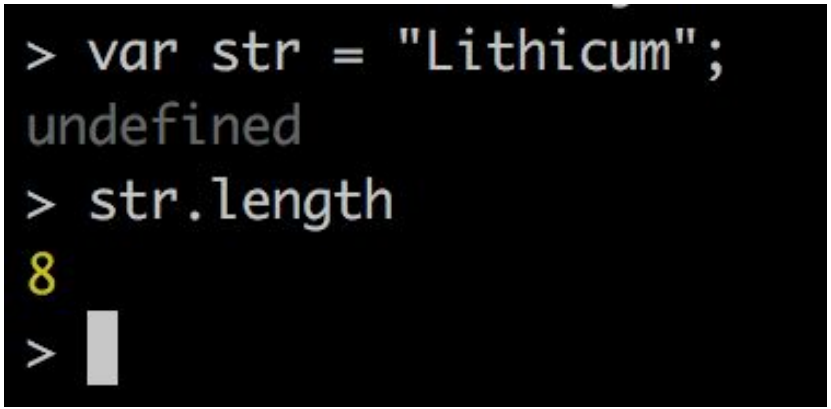
Data Types: String

- Strings are simply a series of characters (or words)
  - JavaScript does not have characters
  - It is perfectly valid to use both single and double quotes for Strings
  - Node actually represents String internally using single quotes
- Strings are 2 bytes per character
  - ‘Hello’ would be 10 bytes

```
> var str = "Hello";
undefined
> str
'Hello'
> var str2 = 'Hello';
undefined
> str2
'Hello'
> str == str2
true
```

String: Properties - length

- String properties will return specific data from the String it was invoked on
- The only one we need to focus on is the ‘length’ property
  - length returns the number of characters in a given String.
  - Note: This includes spaces



String: Functions + Indexes

- These functions perform some sort of action on a given String.
- They are invoked the exact same way as properties using the dot operator.

- As you may already know, indexing always starts at the number 0 and continues to length-1
- Example. If I have the String ‘Hello’
  - ‘H’ would be at index 0
  - ‘e’ would be at index 1
  - ‘o’ would be at index 4 (which is length - 1)

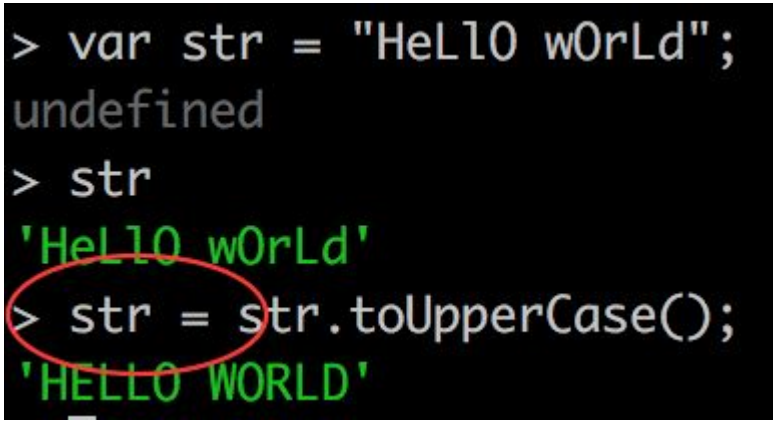
String: Functions - Upper and Lowercase

- It is very common that an input String can have varying cases and a developer would want to convert the entire String to the same case.
- The simple functions toUpperCase() and toLowerCase() solve this problem.

```
> var str = "HeLlO wOrLd";
undefined
> str
'HeLlO wOrLd'
> str = str.toUpperCase();
'HELLO WORLD'
```

String: Functions - Upper and Lowercase (note)

- It is easy to forget to set the value the upper or lowercase conversion equal to a variable. (see annotation below)
- If there was no “str =” portion, then the String would remain in its original state.



String: Functions - substr(start, end)

- The function “substr()” allows a developer to extract a String from within a String.
- This function takes in two parameters:
  - The first: A start index
  - The second: An end index
- *Remember that indexing always starts at zero*

String: Functions - substr() Examples

```
> var str = "Hello world!";
undefined
> var res = str.substr(1, 4);
undefined
> str
'Hello world!'
> res
'ello'
```

```
> var str = "Test";
undefined
> var res = str.substr(0,8);
undefined
> res
'Test'
```

String: Functions - charAt(index)

- The charAt() function returns the character at a given index.
- Note: What happens if the index you provide is outside of the String's bounds?
    - I.e. if the String is “Hello” and we try to look for the character at index 9?

String: Functions - charAt() Examples

```
> var str = "Hello";
undefined
> str
'Hello'
> str.charAt(0);
'H'
```

```
> var str = "Hello";
undefined
> str
'Hello'
> str.charAt(9);
''
```

String: Functions - replace(string1, string2)

- The replace function will search for string1 inside of the provided String, if it is found, then it will replace string1 with string2.
- The given parameters:
  - String1: The String that will be replaced
  - String2: The String that will be the substitute
- Note 1: This is case sensitive!!!
- Note 2: If string1 (the first parameter) is not found, then nothing happens!!!

String: Functions - replace Examples

```
> var str = "Hello World";
undefined
> str
'Hello World'
> str = str.replace("World", "AIT");
'Hello AIT'
> str
'Hello AIT'
```

```
'Hello AIT'
> str = str.replace("Test", "Nothing");
'Hello AIT'
```

String: Functions - indexOf(string)

- indexOf() will return the first index of the String found
- If the String in the parameter does not exist, then -1 is returned

- Note 1: This is also case sensitive!!

String: Functions - indexOf() Examples

```
> var str = "Hello World";
undefined
> str
'Hello World'
> str.indexOf('H');
0
> str.indexOf('ello');
1
> str.indexOf('l');
2
> str.indexOf('x');
-1
```

String: Functions - split(string)

- The split() method is used to split a string into an array of substrings, and returns the new array.
    - We will go into arrays later in the lecture, but is necessary for using split

- Note 1: If an empty string ("") is used as the separator, the string is split between each character.
- Note 2: Like most of these functions, it does not alter the original String.
- *wink wink* might be important for assignment 1 part 2.

String: Functions - split() Examples

```
> var str = "Welcome to AIT 618!";
undefined
> var arr = str.split(' ');
undefined
> arr
[ 'Welcome', 'to', 'AIT', '618!' ]
```

```
> var arr2 = str.split('');
undefined
> arr2
[ 'W',
  'e',
  'l',
  'c',
  'o',
  'm',
  'e',
  ' ',
  't',
  'o',
  ' ',
  'A',
  'I',
  'T',
  ' ',
  '6',
  '1',
  '8',
  '!' ]
```

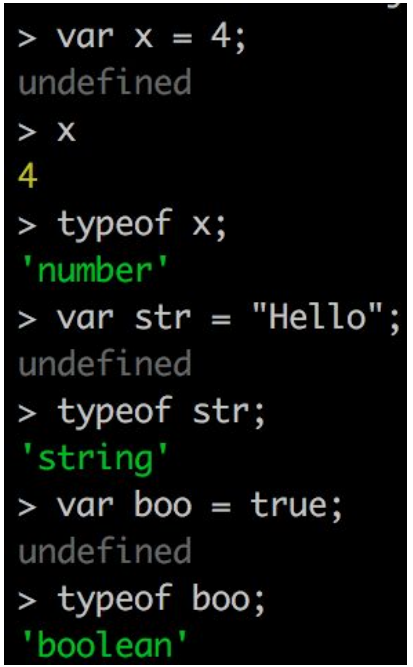
Final split() Example *wink wink*

- Let’s say we had a text file we are reading in. When reading a text file in JavaScript, it is represented as Strings separated by newlines
  - Newlines are represented as the character ‘\n’

- ‘This is a\ntextfile’
  - The above could potentially be the String that the textfile is read as
- So: str.split(‘\n’) would give us [‘This is a’, ‘texfile’];
  - It is an easy way to count the number of newlines in a textfile (length - 1).

typeof Operator

- Since we talked about Strings, Numbers, Booleans, null, etc. there is an operator that tells us the Data Type of a given variable.



Data Structures

Data Structures: Definition

- Data Structures are simply a way of organizing variables into one so it can be used efficiently.
- The two most important ways of organizing data in this class will be using:
  - Arrays
  - JSON

Data Structures: Arrays

- Arrays are considered a linear data structure
- This means that each element is stored one after the other
- Just like indexing the characters in a String, Arrays work the same way
  - Starting at 0 and going to length - 1
- Arrays are defined by using brackets ‘[ ]’
  - var x = [1,2,3,4,5];
  - var x = [‘hello’, ‘welcome’, ‘to’, ‘AIT 618!’];

Data Structures: Arrays (cont.)

- Notice how the left hand-side of the assignment is the exact same as if you were going to declare a String, Number, or Boolean
- The ‘loosely-typed’ paradigm applies to arrays as well
- There is also no need to specify how large the array will be, all of it is dynamic.

```
> var x = [1,2,3,4];
undefined
> x
[ 1, 2, 3, 4 ]
```

Arrays: Properties - length

- Similarly to Strings, an array contains a ‘length’ property which returns the number of elements in an array.

```
> var x = [1,2,3,4];
undefined
> x
[ 1, 2, 3, 4 ]
> x.length;
4
```

```
> var strs = ['how', 'long', 'is', 'this', 'array'];
undefined
> strs
[ 'how', 'long', 'is', 'this', 'array' ]
> strs.length;
5
```

Arrays: Functions

- Since arrays in JavaScript are dynamic and there is no need to specify size, three functions have a massive impact on array manipulation.
- push()
- pop()
- concat()

Arrays: Functions (cont.)

- Arrays in JavaScript are represented very similar to the way a stack is represented
  - But don’t worry, it is not a stack!

Arrays: Functions - push()

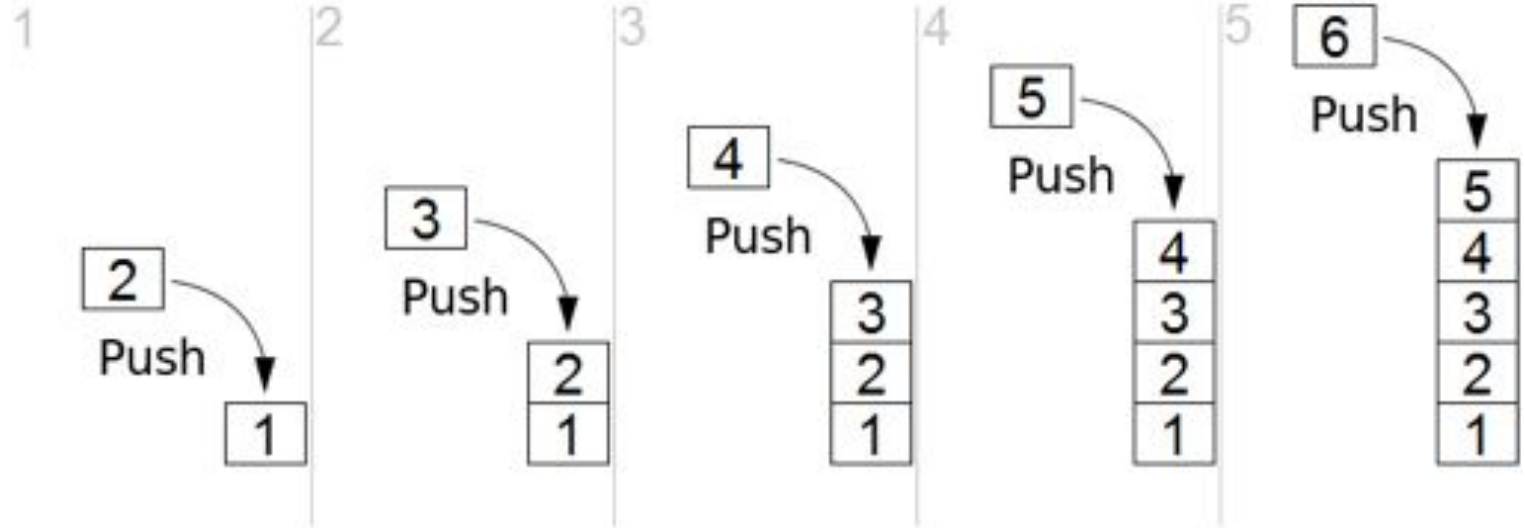
- The push() function allows you to add an element to the end of the array

```
> var arr = ["hello", "welcome", "to"];
undefined
> arr
[ 'hello', 'welcome', 'to' ]
> arr.push("AIT");
4
> arr
[ 'hello', 'welcome', 'to', 'AIT' ]
```

Arrays: Functions - push() Elucidated

- var arr = [1];
  - arr.push(2);
  - arr.push(3);
  - arr.push(4);
  - arr.push(5);
  - arr.push(6);

Arrays: Functions - push (visual)



Arrays: Functions - pop()

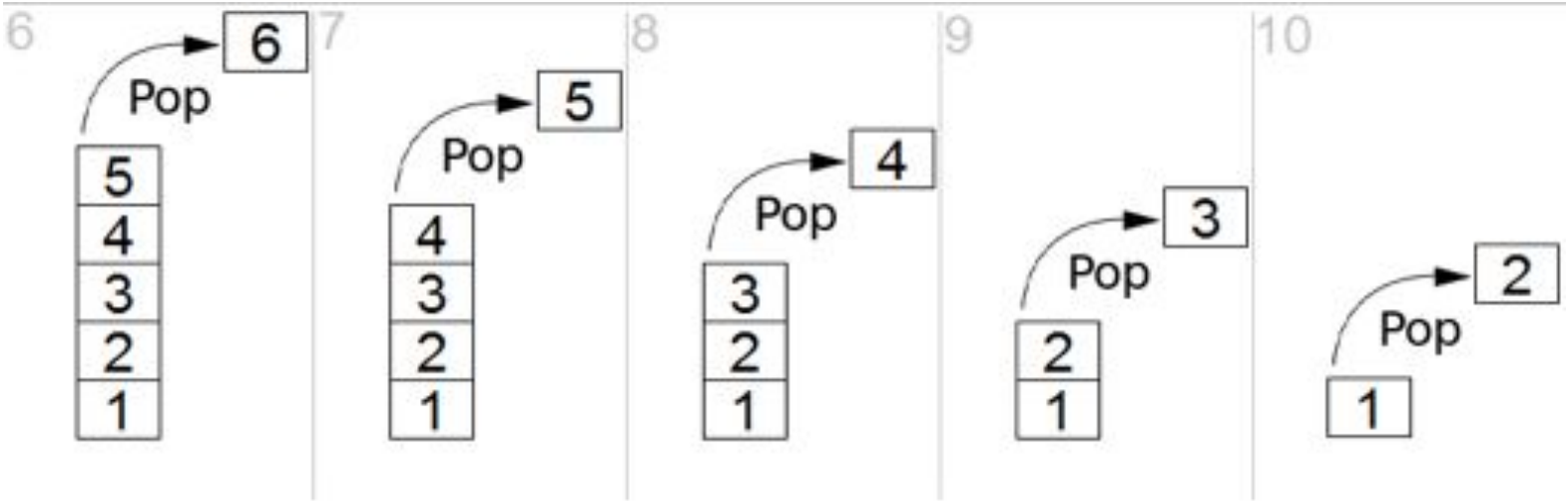
- The pop() function will remove the last element
  - Note pop() returns a value!
  - Could be useful in the next assignment

```
> arr
[ 'hello', 'welcome', 'to', 'AIT' ]
> arr.pop();
'AIT'
> arr
[ 'hello', 'welcome', 'to' ]
```

Arrays: Functions - pop() Example

- var arr = [1,2,3,4,5,6];
  - arr.pop();
  - arr.pop();
  - arr.pop();
  - arr.pop();
  - arr.pop();

Arrays: Functions - pop() (visual)



Arrays: Functions - pop()

- If we had the array: var arr = [1,2,3,4];
- Then we did var x = arr.pop();
  - x would be 4
  - arr would be [1,2,3]

Arrays: Functions - concat()

- The concat() function is a way to combine two arrays into one
  - var x = [1,2,3,4];
  - var y = [5,6,7,8];
  - var z = x.concat(y);
  - z = [1,2,3,4,5,6,7,8];
- What would the array look like if I did “var z = y.concat(x);”?

Arrays: Functions - concat() Example

- Note that concat() returns an array of the two arrays combined

```
> var arr1 = ["Welcome", "to", "baltimore"];
undefined
> var arr2 = ["Home", "of", "the", "ravens"];
undefined
> arr1
[ 'Welcome', 'to', 'baltimore' ]
> arr2
[ 'Home', 'of', 'the', 'ravens' ]
> var arr3 = arr1.concat(arr2);
undefined
> arr3
[ 'Welcome', 'to', 'baltimore', 'Home', 'of', 'the', 'ravens' ]
```

Node.js Global Object - process

- Process is a global object built into the Node.js namespace
- It’s function is to contain all static data for a given node process
- The most important feature about this is the process.argv[] array.
- Since we have been running our Node.js projects via the commandline, process.argv[] reads in all of the commands that are typed into the terminal when the program is run
    - process.argv[0] has the first command
    - Process.argv[1] has the second command
    - etc.

Node.js Global Object - process Example

process.js

```
1  console.log("First Command: " + process.argv[0]);
2  console.log("Second Command: " + process.argv[1]);
3  console.log("Third Command: " + process.argv[2]);
```

```
Jals-MacBook-Pro:Desktop jalirani$ node process.js testArguement
First Command: /Users/jalirani/.nvm/versions/node/v8.1.3/bin/node
Second Command: /Users/jalirani/Desktop/process.js
Third Command: testArguement
```

process.argv[] Notes

- Notice that:
  - Index 0 will almost always be the path to node
  - Index 1 will almost always be the path to your file
  - All other optional arguments will be strings that you typed in

JSON

- Another way to store data is through an object type called JSON, which stands for:
  - JavaScript
  - Object
  - Notation
- In all modern servers and APIs, this is the format that is returned after most requests (mainly HTTP)

JSON - format

- JSON has the following format:
  - {key: value}
- The ‘key’ is always a String, since it must be unique
  - It does not make sense to have an number or boolean as a key and we’ll see why when you try to access values
- The ‘value’ can be any datatype, even an array or another JSON object!

JSON - format Example

- Below is an example of a simple JSON object (or blob). Ignore the '...' that is the REPL way of saying you have not closed off the statement.

```
> var jsonBlob = {
... 'fName': 'jal',
... 'lName': 'irani',
... 'age': 25
... }
undefined
> jsonBlob
{ fName: 'jal', lName: 'irani', age: 25 }
```

File I/O

Synchronous File I/O

- Node.js is built on the asynchronous paradigm meaning that node does not wait for tasks to complete.
  - Aka there are is no “blocking” I/O
- In two weeks you’ll learn Async File I/O, but today you’ll learn the much simpler Synchronous File I/O
  - I.e. will wait for entire file to be read before executing any code

FileSystem Module in Node.js

- First Modules are imported just to make life easier
  - Just like in Java “import Scanner”
  - Or C with #include <stdio.h>
- The FileSystem module allows you to read and write to files
- Can be included using:
  - var fs = require(‘fs’);
  - Normally placed at the top of the file
- Note: the module is set equal to the variable ‘fs’. The module is now contained inside of the variable and we can use the variable wherever we’d like.

fs Installation

- Type ‘npm install fs’ 

- Note: you may need to ‘sudo npm install fs’ depending on how your security preferences are set.
    - You may not need to npm install fs any longer depending on node version

FileSystem Module in Node.js

- Now that we have fs installed we can use it
- var buffer = fs.readFileSync('fileName.txt');
  - This will read fileName.txt in the current directory and store as a buffer in the var called 'buffer'
- Buffers are a series of hexidecimal numbers which are not human readable.
- Always convert a buffer to a String by doing:
  - var output = buffer.toString();